

COMPUTER ARCHITECTURE



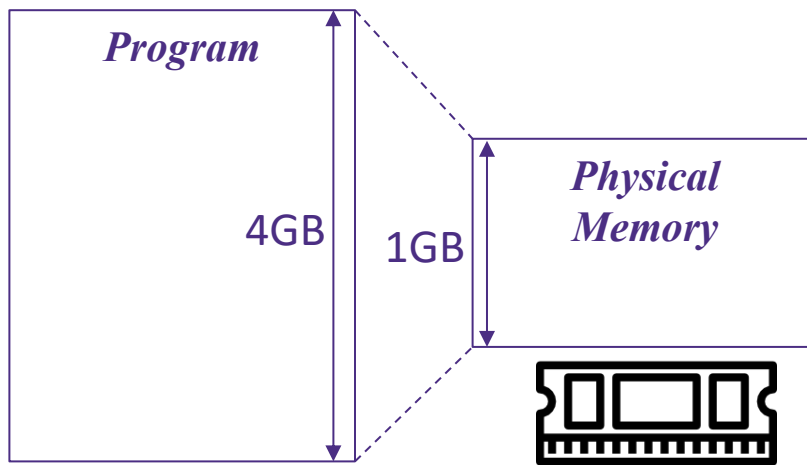
LECTURE 13: Virtual Memory

CONTENTS

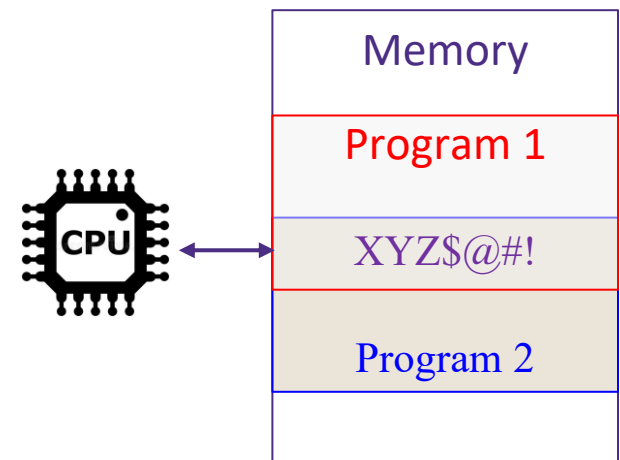
- > Introduction
 - Core concepts
- > Address translation
 - The TLB (Translation Look-aside Buffer)
- > The scalability issue
 - Multi-level page tables
- > The OS policy
 - Page replacement

Introduction

- ❑ Modern computers can run multiple programs simultaneously
 - P1: How can we run programs **exceeding** physical memory?
 - P2: How can we prevent each program from **interfering** with other's memory?



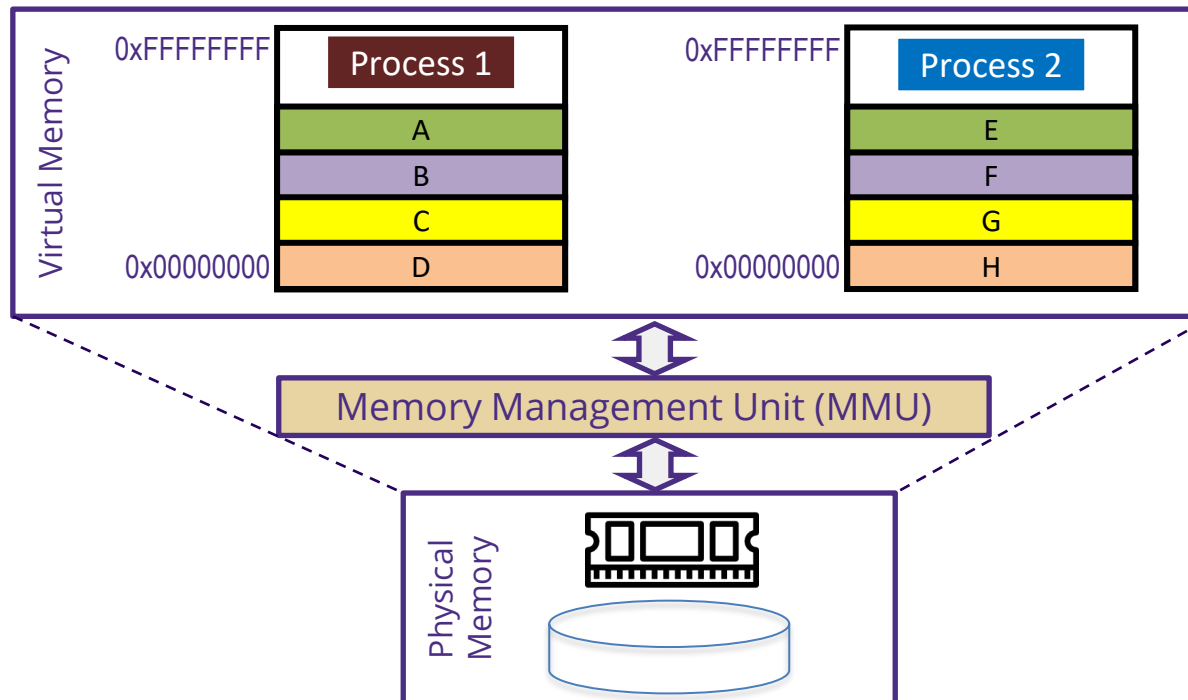
⚠ Crash if we try to access an address
> **0x3FFF FFFF!**



⚠ Corrupt if each process can
access any memory address

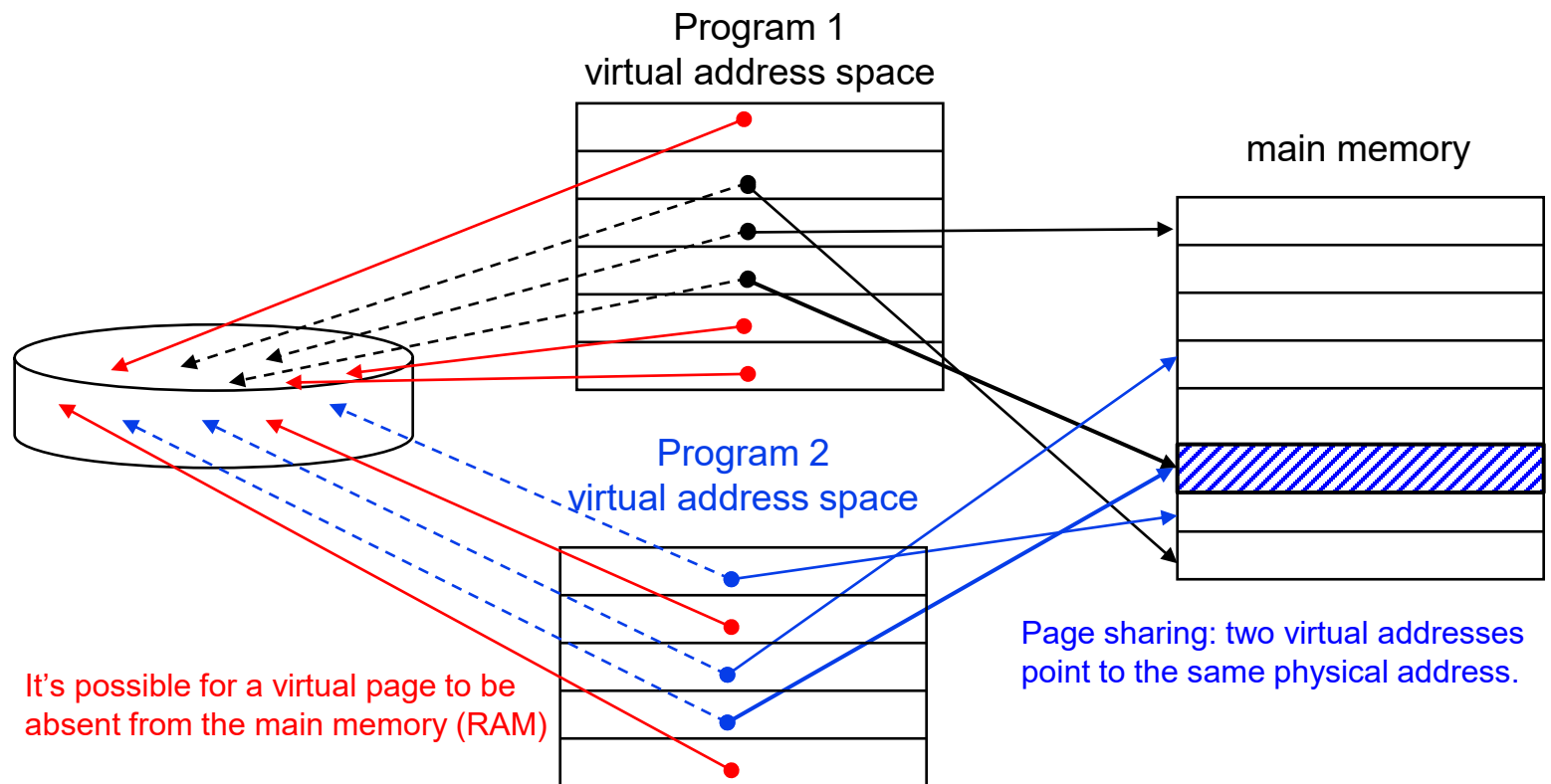
Virtual Memory

- ❑ Solution: Virtual memory - the next level in the memory hierarchy
 - Giving each process the **illusion** of its **own full address space**
 - Hiding the actual physical memory layout from processes
 - Storing active data (**working set**) in RAM while keeping inactive data on disk



Virtual - Physical Memory Mapping

- ❑ VA space is mapped onto PA space via **paging**
 - Memory is divided into fixed blocks called page (virtual) & frame (physical)
 - MMU translates "Virtual Addresses" (fake) to "Physical Addresses" (real)



Physical Address Space

❑ Divided into equal sized **page frames**

- Typically 4KB each

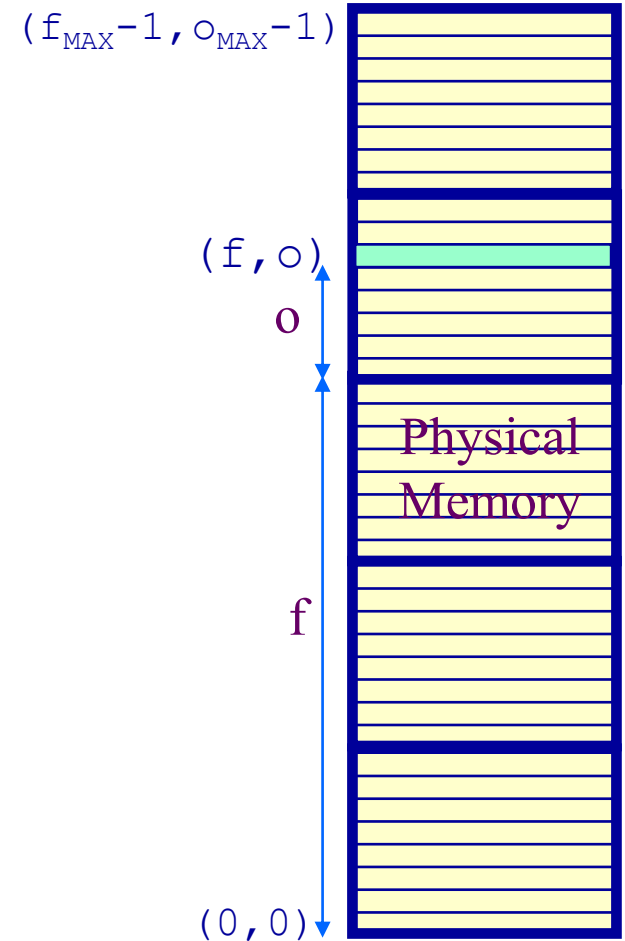
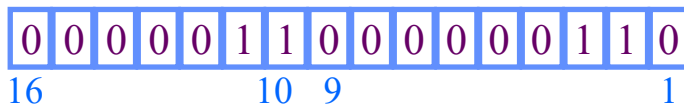
❑ A PA is specified by a pair (f, o)

- f : frame number ($f_{\max}$ frames)
- o : frame offset ($o_{\max}$ bytes/frames)
- Physical address = $o_{\max} \times f + o$

❑ **Example:** Find the PA if

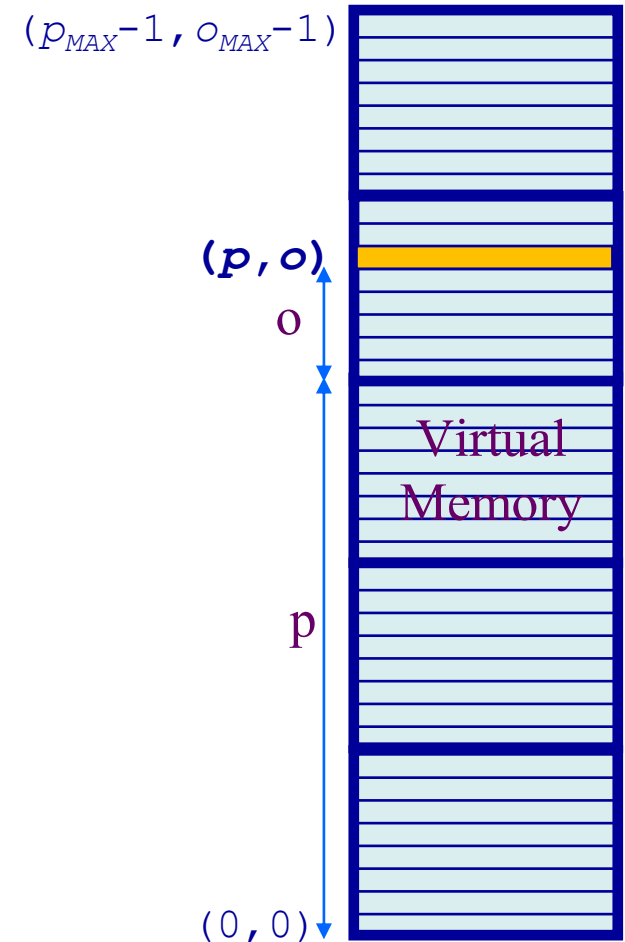
- 16-bit address
- $o_{\max} = 512$ bytes/frames
- addressing location (3, 6)

❑ **Solution:**



Virtual Address Space

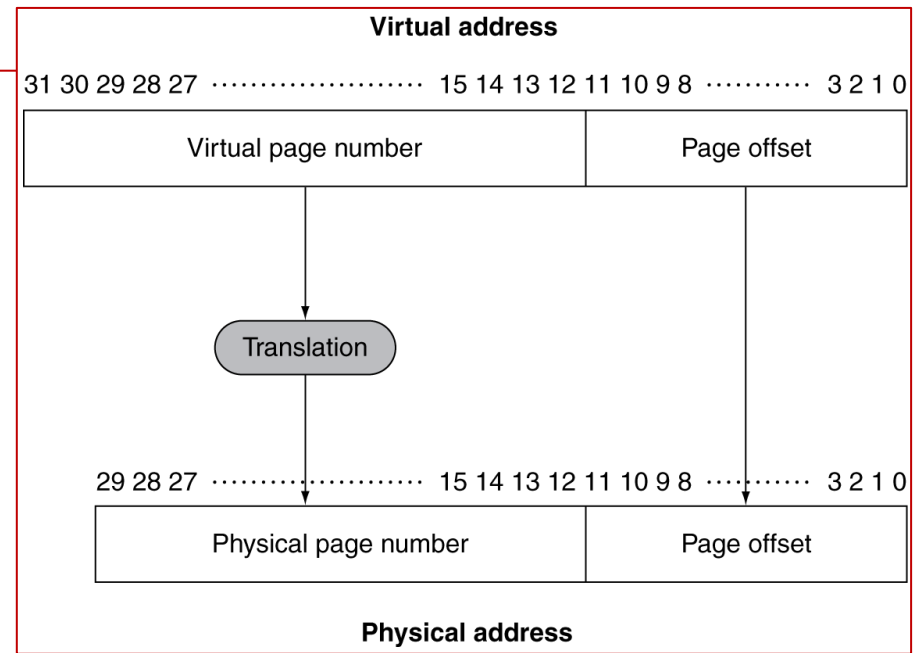
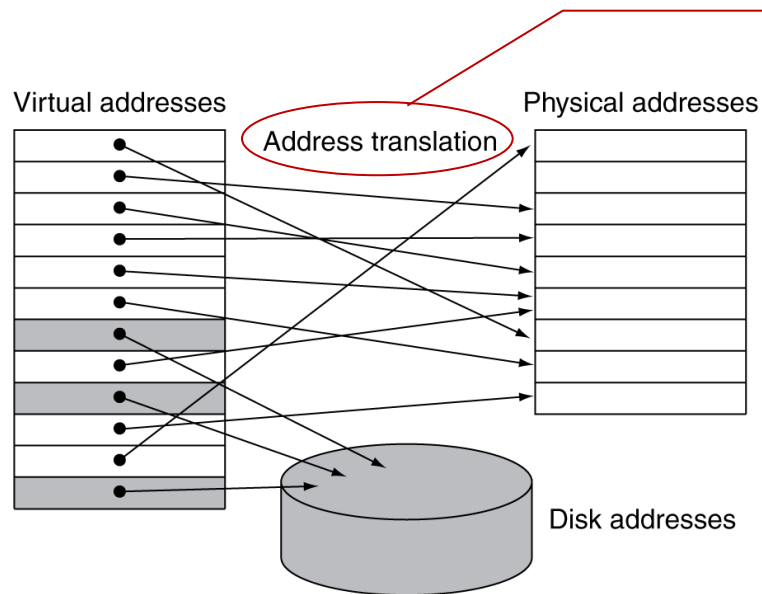
- ❑ Divided into equal sized **pages**
 - same size as page frames
- ❑ A VA is specified by a pair (p, o)
 - p : page number ($p_{\max}$ pages)
 - o : page offset ($o_{\max}$ bytes/pages)
 - Virtual address = $o_{\max} \times p + o$
- ❑ Important notices:
 - $o_{\max}$ is the same for both pages and frames
 - normally $p_{\max} > f_{\max}$
- ❑ Why fixed-size Pages?
 - Eliminates external fragmentation
 - Simplifies memory allocation
 - Makes address translation straightforward



Address Translation

□ Mapping: [VPN | Offset] → [PPN | Offset]

- The "Offset" never changes: translation = mapping a VPN to a PPN
- Result of a translation can point to a page frame in DRAM or a block on disk



The Page Table

❑ Where does the MMU find the mapping VPN → PPN? **Page table**

- Each process has a dedicated page table
- Page table itself resides in **main memory**
- E.g. $2^{32}/2^{12} = 2^{20}$ (1M) entries

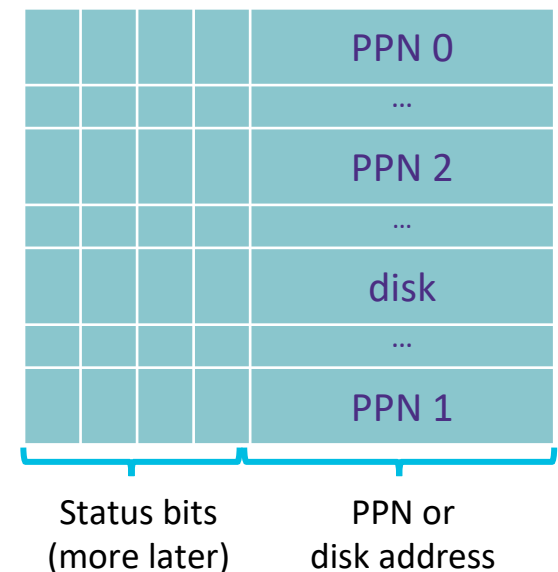
❑ Page table contents:

- One entry per virtual page number.
- Entry has physical page number (or disk address) as well as status bits.

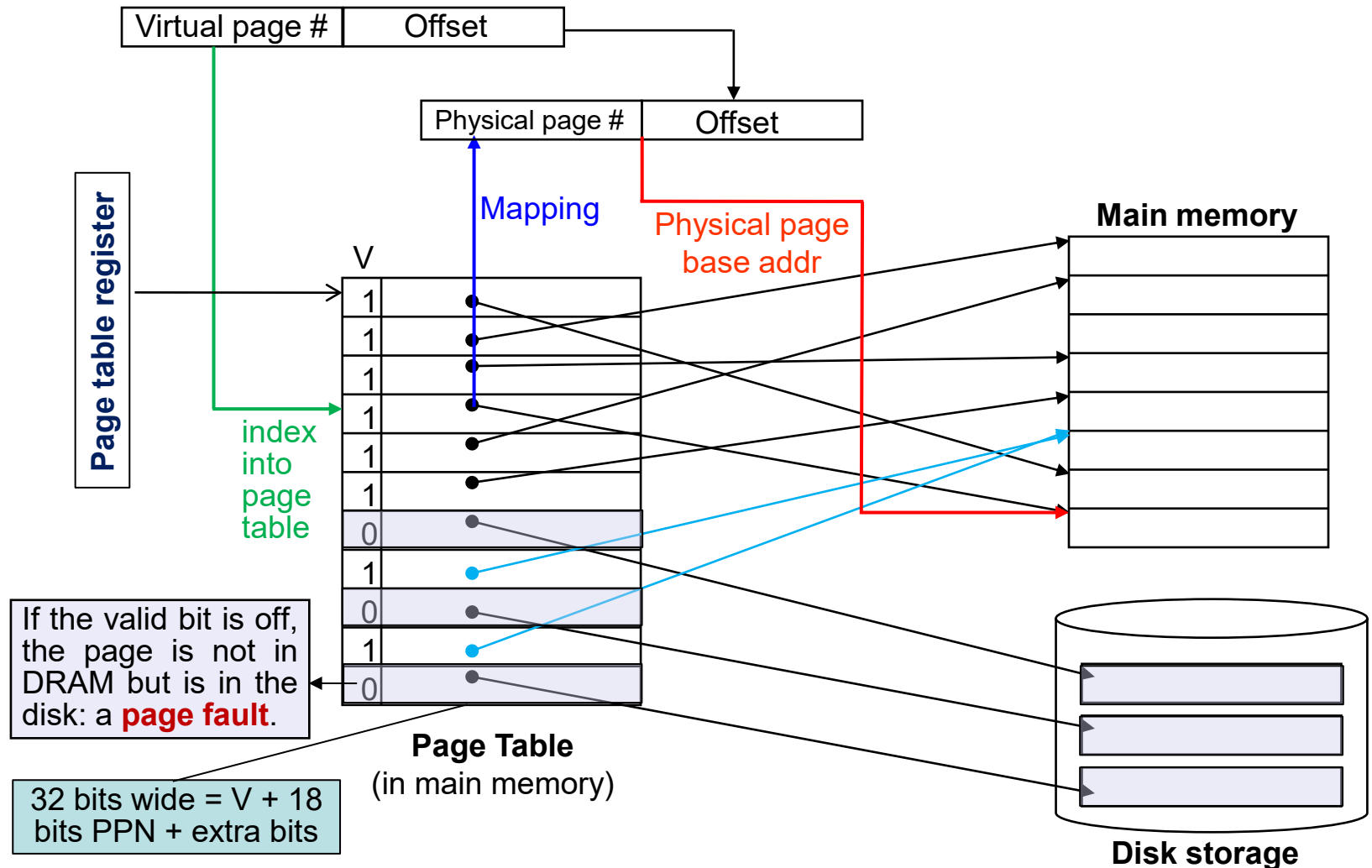
❑ Page table is **NOT** a cache!!

- Page tables don't hold the program's data: it's just a lookup table.
- Each VPN has a valid entry: no tags, the VPN is used as an index.

Page Table
(2^{20} entries)

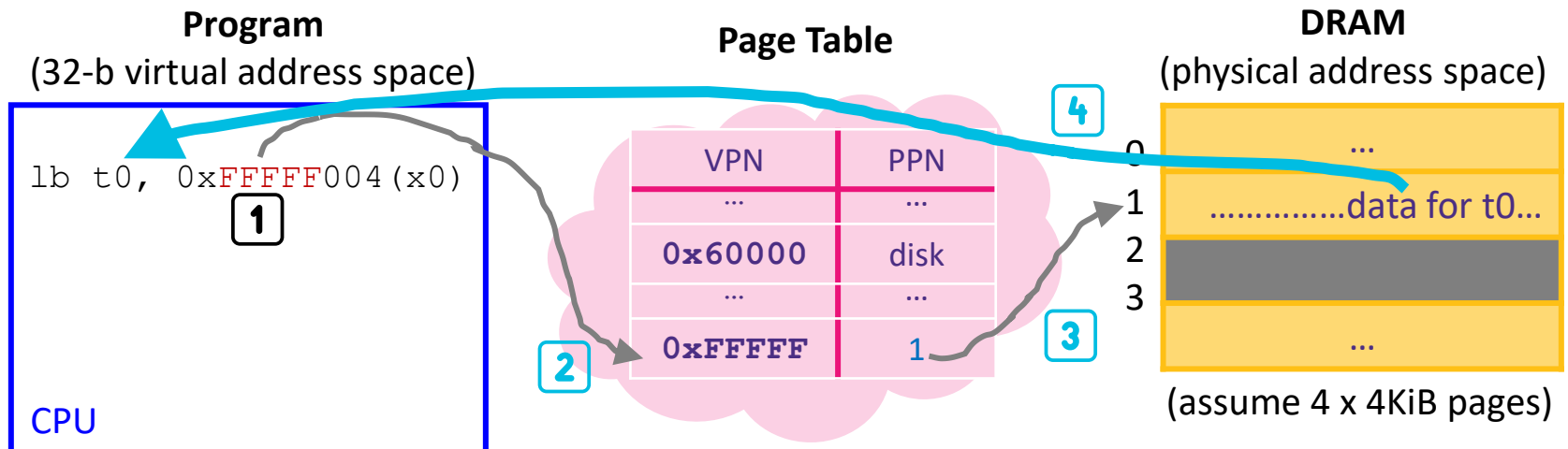


The Translation Process



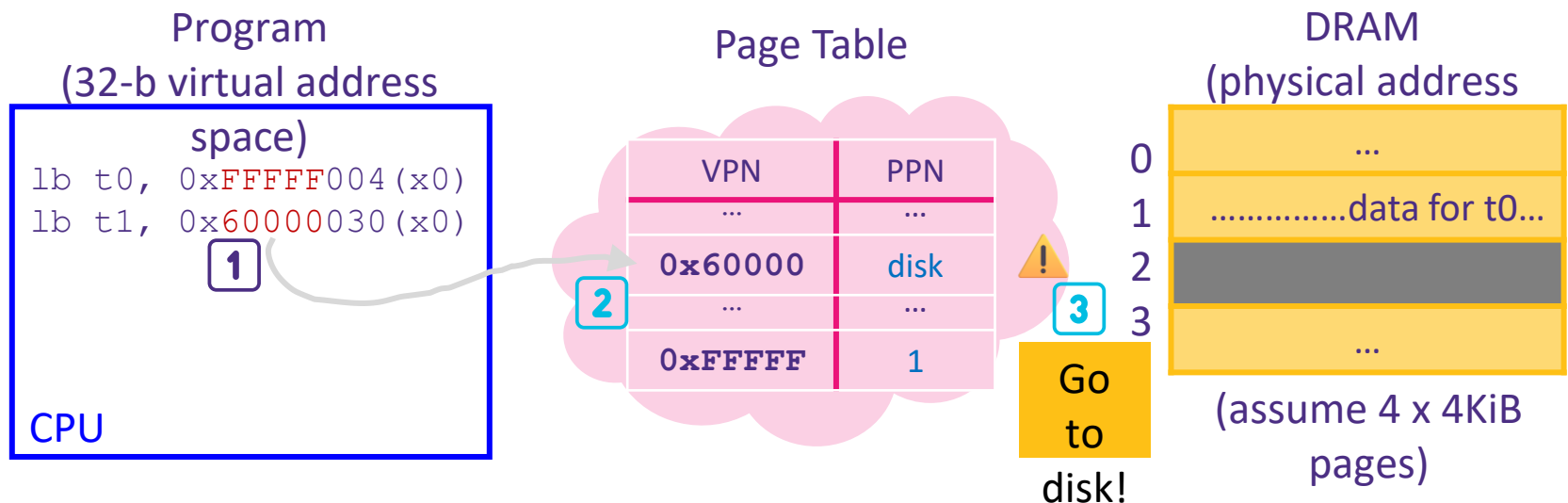
Example: Page Hit

1. Program executes a load specifying a virtual address (VA).
2. Computer translates VA to the physical address (PA) in memory
 - Extract virtual page number (VPN) from VA, e.g. top 20 bits if page size 4KiB
 - Look up the physical page number (PPN) in the page table.
3. Page hit: $PA = PPN \text{ (from page table)} + \text{offset (from VA)}$
 - OS reads memory & returns the data to the program.



Example: Page Fault

1. Program executes a load specifying a virtual address (VA).
2. Computer translates VA to the physical address (PA) in memory
 - Extract virtual page number (VPN) from VA, e.g. top 20 bits if page size 4KiB
 - Look up the physical page number (PPN) in the page table.
3. **Page fault:** the OS uses the **swap table** to find the disk block
 - OS reads the disk, loads the data to RAM & returns it to the program.



Page Fault Management

- ❑ Occurs when the “reference bit” is 0 (indicating data is on disk)
 - HW and OS work together to manage page fault
 - Hardware raises an exception ("Page Fault")
 - OS handles the exception
- ❑ The OS handler:
 1. Pause the process
 2. Find the data on the HDD/SSD (via swap table)
 3. Find an empty frame in RAM (or kick someone out)
 4. Copy data HDD/SSD → RAM (using DMA)
 5. Update page table (reference bit = 1)
 6. Retry the instruction
- ❑ Service time for a page fault is ~ millions of cycles
 - Unavoidable with **demand paging**

Demand Paging

- ❑ OS only loads essential pages into RAM when a process starts
 - Helps reduce initial load time and memory footprint
 - Other pages are loaded into RAM when needed through page fault
- ❑ Paging performance: **Effective Memory Access Time (EAT)**
 - $EAT = \text{memory access time} \times \text{probability of a page hit} + \text{page fault service time} \times \text{probability of a page fault}$
- ❑ **Example:**
 - Memory access time: 60 ns
 - Disk access time: 25 ms
 - Let p = the probability of a page fault
 - To realize an EAT within 5% of minimum, what's the largest value of p ?
- ❑ **Solution:**
 - Require $EAT \leq 1.05 \times 60 = 63 \text{ ns}$
 - $EAT = 60(1-p) + 25,000,000p \leq 63 \rightarrow p \leq 1.2 \times 10^{-7}$

Making Address Translation Fast

- ❑ A virtual memory request requires extra memory references
 - one to translate Virtual Address into Physical Address (page table lookup)
 - one to transfer the actual data (hopefully cache hit).

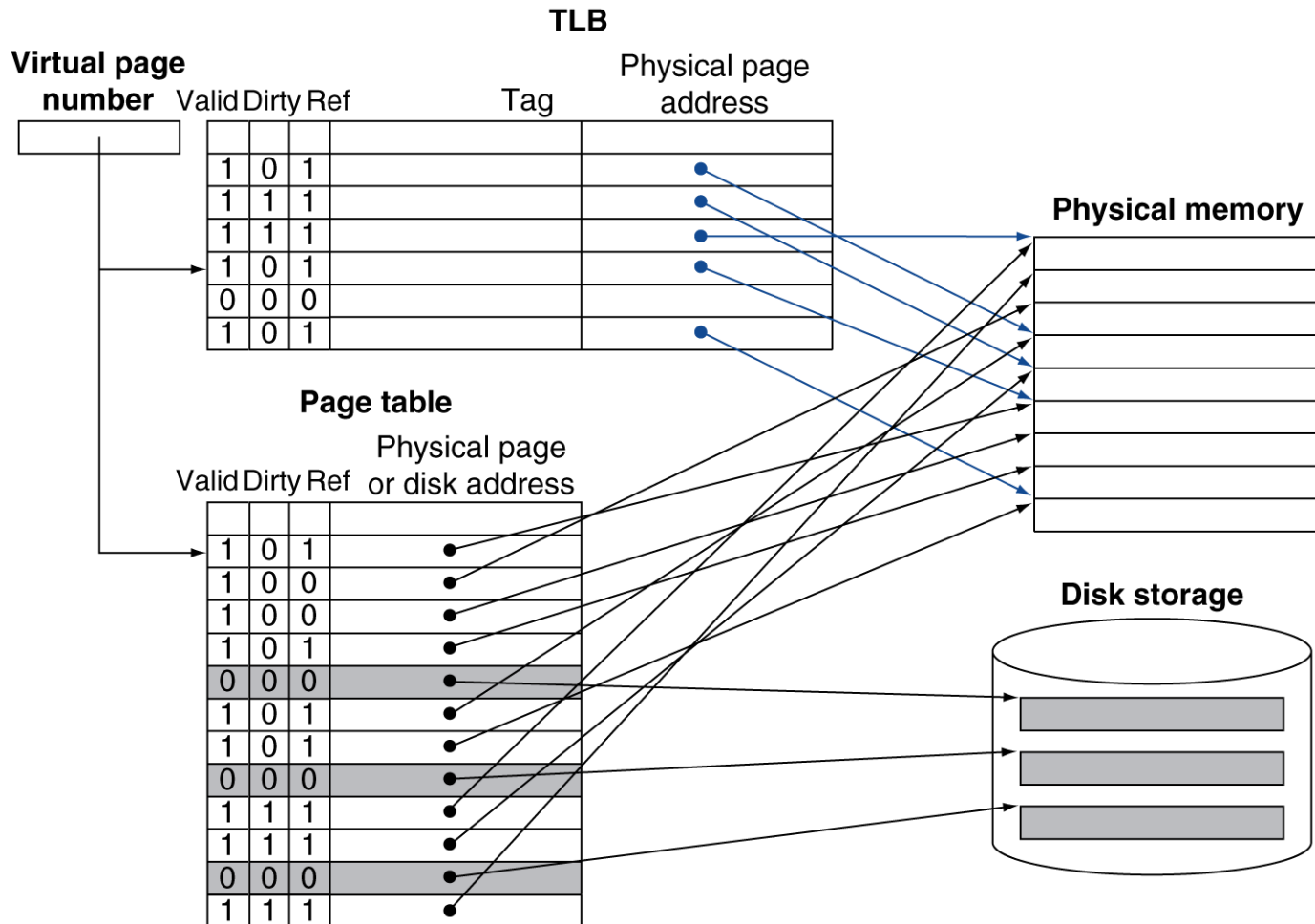
- ❑ But access to page tables has good locality

- Use a **fast cache** of page tables inside the MMU

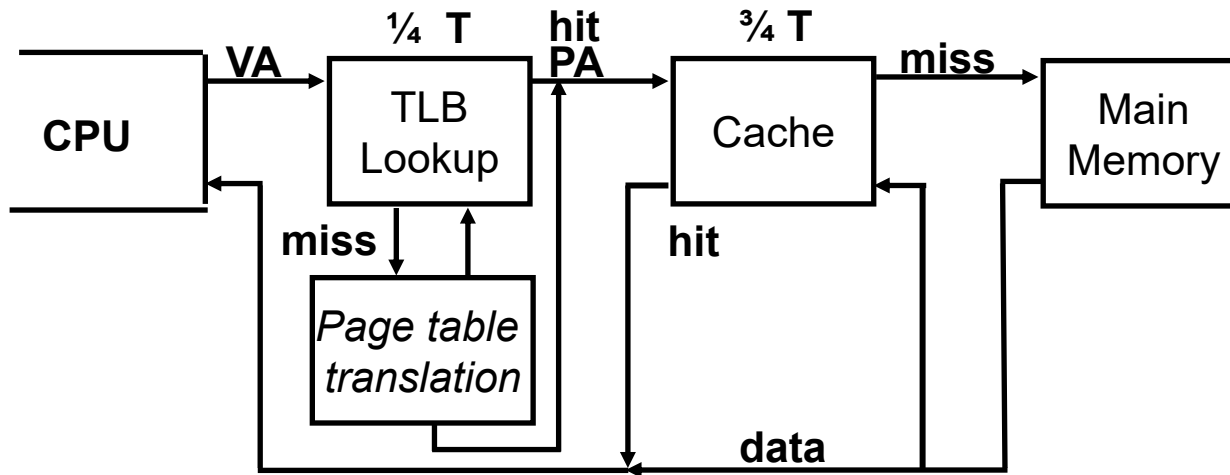
V	Dirty	Ref	Tag (VPN)	PPN

- Called a **T**ranslation **L**ook-aside **B**uffer (**TLB**)
- Typical design: fully associative, 16–512 entries, 0.5–1 cycle for hit, 10–100 cycles for miss, 0.01%–1% miss rate, random/FIFO replacement policy.
- Misses could be handled by hardware or software.

TLB Implementation



Full Memory Hierarchy

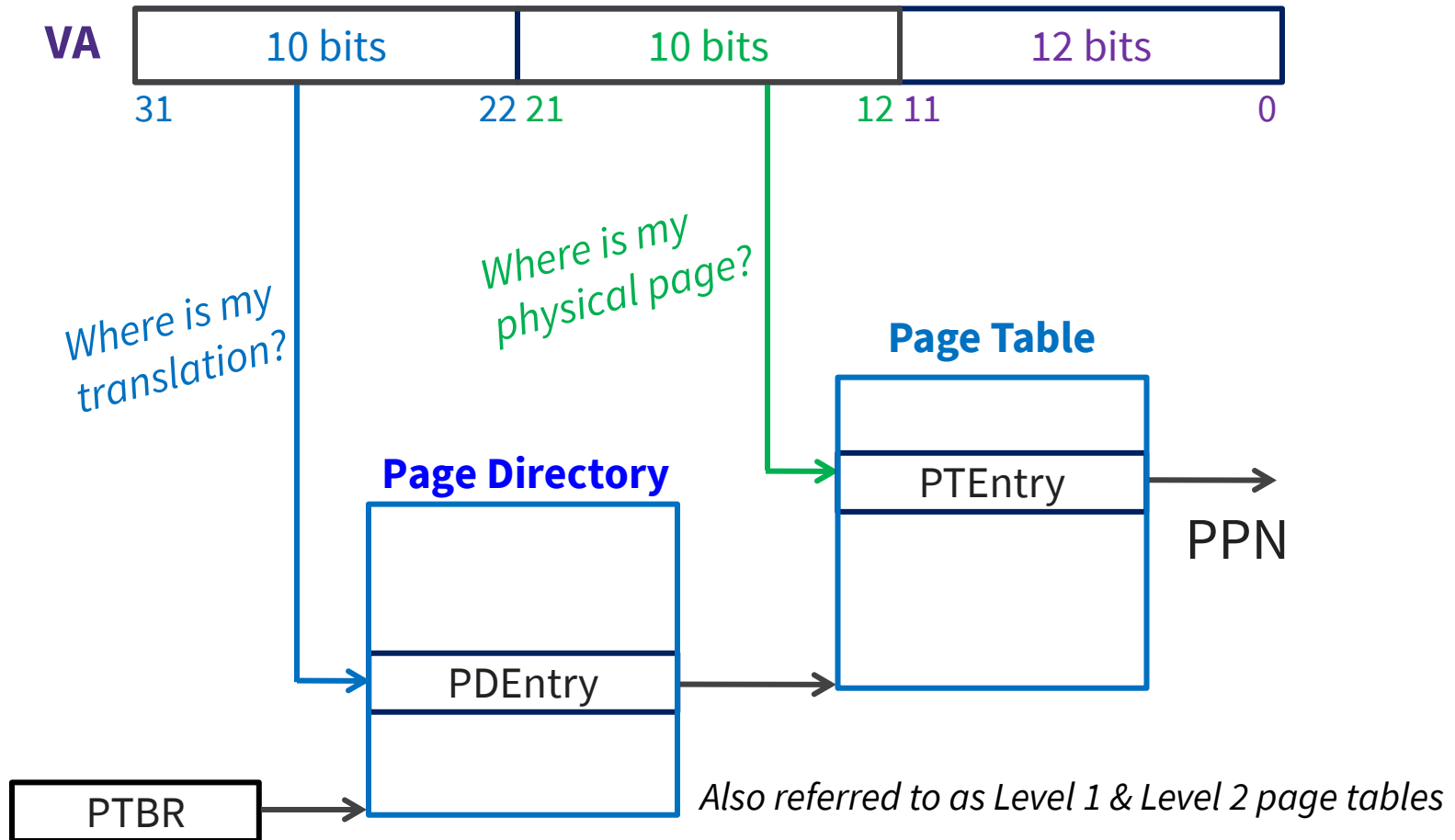


- ❑ A TLB miss – is it a page fault or merely a TLB miss?
 - No, if the page is already loaded into main memory → finds & loads the information from the page table into the TLB (takes ~10's of cycles).
 - Yes, if the page is not in main memory → it's a true page fault (takes ~millions of cycles to service a page fault).
- ❑ TLB misses are much more frequent than true page faults

The Space Problem

- ❑ With 4GB virtual address space and 4KB pages:
 - 1 million page table entries per process
 - Entry size = 4 Bytes
 - 4MB per process just for the page table
 - Multiple processes → page tables consume too much memory
- ❑ Solution: Multi-level page table
 - Break the table into a tree (Directory → Table → Page)
 - If a large chunk of memory is unused, the "Page Directory" marks it as Null
 - We do not allocate the sub-tables for that empty space
- ❑ Trade-off:
 - ✓ A process might require only 2 small tables, no need for a contiguous 4MB table
 - ✗ Longer lookup time (multiple memory accesses)
 - ✗ Slightly more memory if all pages used (due to multi-level overhead)

Example: Two-level Page Table



Page Replacement Policies

- ❑ If RAM is full and a process faults
 - Some page must be evicted to make room. Which page should be replaced?
- ❑ Clairvoyant:
 - Replace the page that won't be needed for the longest time
 - Optimal but impractical: Can't look forward in time
- ❑ FIFO (First-In, First-Out): Simple but bad performance
 - Can throw out heavily used pages just because they are old
- ❑ LRU (Least Recently Used): “Back to the future” heuristic
 - Expensive HW implementation: Requires timestamping every access
- ❑ The clock replacement: LRU approximation
 - Use a simpler approach instead of maintaining exact access times

Example: LRU Replacement

Time		0	1	2	3	4	5	6	7	8	9	10
Requests			c	a	d	b	e	b	a	b	c	d
Page Frames	0	a	a	a	a	a	a	a	a	a	a	a
	1	b	b	b	b	b	b	b	b	b	b	b
	2	c	c	c	c	c	e	e	e	e	e	d
	3	d	d	d	d	d	d	d	d	d	c	c
Faults							•				•	•
Time page last used						a = 2 b = 4 c = 1 d = 3				a = 7 b = 8 e = 5 d = 3	a = 7 b = 8 e = 5 c = 9	

❑ Note: did a bad job at time 9

➤ Replaced a page that we're just going to reference 1 (virtual) time unit later.

LRU Implementation

Time	0	1	2	3	4	5	6	7	8	9	10
Requests		c	a	d	b	e	b	a	b	c	d
Page Frames	0	a	a	a	a	a	a	a	a	a	a
	1	b	b	b	b	b	b	b	b	b	b
	2	c	c	c	c	e	e	e	e	e	d
	3	d	d	d	d	d	d	d	d	c	c
Faults						•				•	•

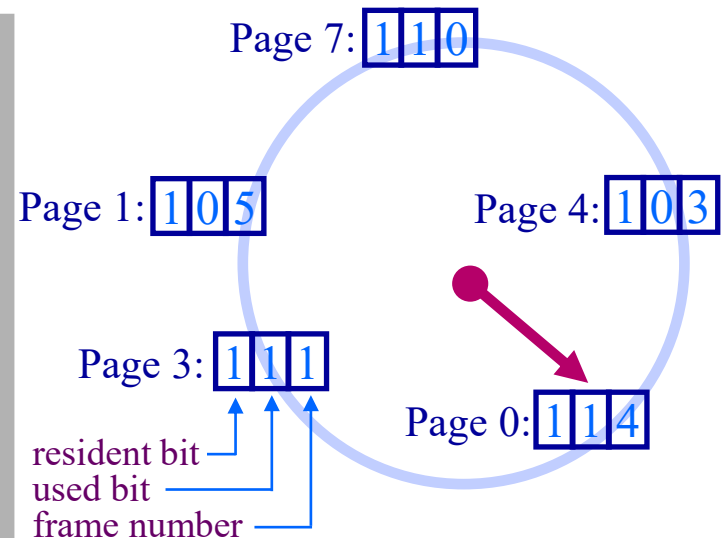
LRU page stack	c	a	d	b	e	b	a	b	c	d
		c	a	d	b	e	b	a	b	c
			c	a	d	d	e	e	a	b
				c	a	a	d	d	e	a
					c				d	e

- ❑ Maintain a “stack” of recently used pages
 - Impractical: stack size **depends** on number of frames allocated to the process

LRU Approximation (The Clock Algorithm)

- ❑ Much simpler to implement than full LRU
 - OS maintains a circular list of all resident pages (the **clock**)
 - HW set the page's **used bit** whenever it is assessed
- ❑ Clock hand sweeps pages and evict the first with use bit = 0
 - Runs every time a page fault occurs, continuing from where it left off last time.

```
func Clock_Replacement
begin
  while (victim page not found) do
    if (used bit for current page = 0) then
      replace current page
    else
      reset used bit
    end if
    advance clock pointer
  end while
end Clock_Replacement
```



Example: Clock Replacement

Time	0	1	2	3	4	5	6	7	8	9	10
Requests		<i>c</i>	<i>a</i>	<i>d</i>	<i>b</i>	<i>e</i>	<i>b</i>	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>
Page Frames	0	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>e</i>	<i>e</i>	<i>e</i>	<i>e</i>	<i>e</i>	<i>d</i>
1	<i>b</i>	<i>b</i>	<i>b</i>	<i>b</i>	<i>b</i>	<i>b</i>	<i>b</i>	<i>b</i>	<i>b</i>	<i>b</i>	<i>b</i>
2	<i>c</i>	<i>c</i>	<i>c</i>	<i>c</i>	<i>c</i>	<i>c</i>	<i>c</i>	<i>a</i>	<i>a</i>	<i>a</i>	<i>a</i>
3	<i>d</i>	<i>d</i>	<i>d</i>	<i>d</i>	<i>d</i>	<i>d</i>	<i>d</i>	<i>d</i>	<i>d</i>	<i>c</i>	<i>c</i>
Faults						•		•		•	•

Page table entries for resident pages:

0	<i>a</i>
0	<i>b</i>
1	<i>c</i>
0	<i>d</i>

1	<i>a</i>
1	<i>b</i>
1	<i>c</i>
1	<i>d</i>

1	<i>e</i>
0	<i>b</i>
0	<i>c</i>
0	<i>d</i>

1	<i>e</i>
1	<i>b</i>
0	<i>c</i>
0	<i>d</i>

1	<i>e</i>
0	<i>b</i>
1	<i>a</i>
0	<i>d</i>

1	<i>e</i>
1	<i>b</i>
1	<i>a</i>
0	<i>d</i>

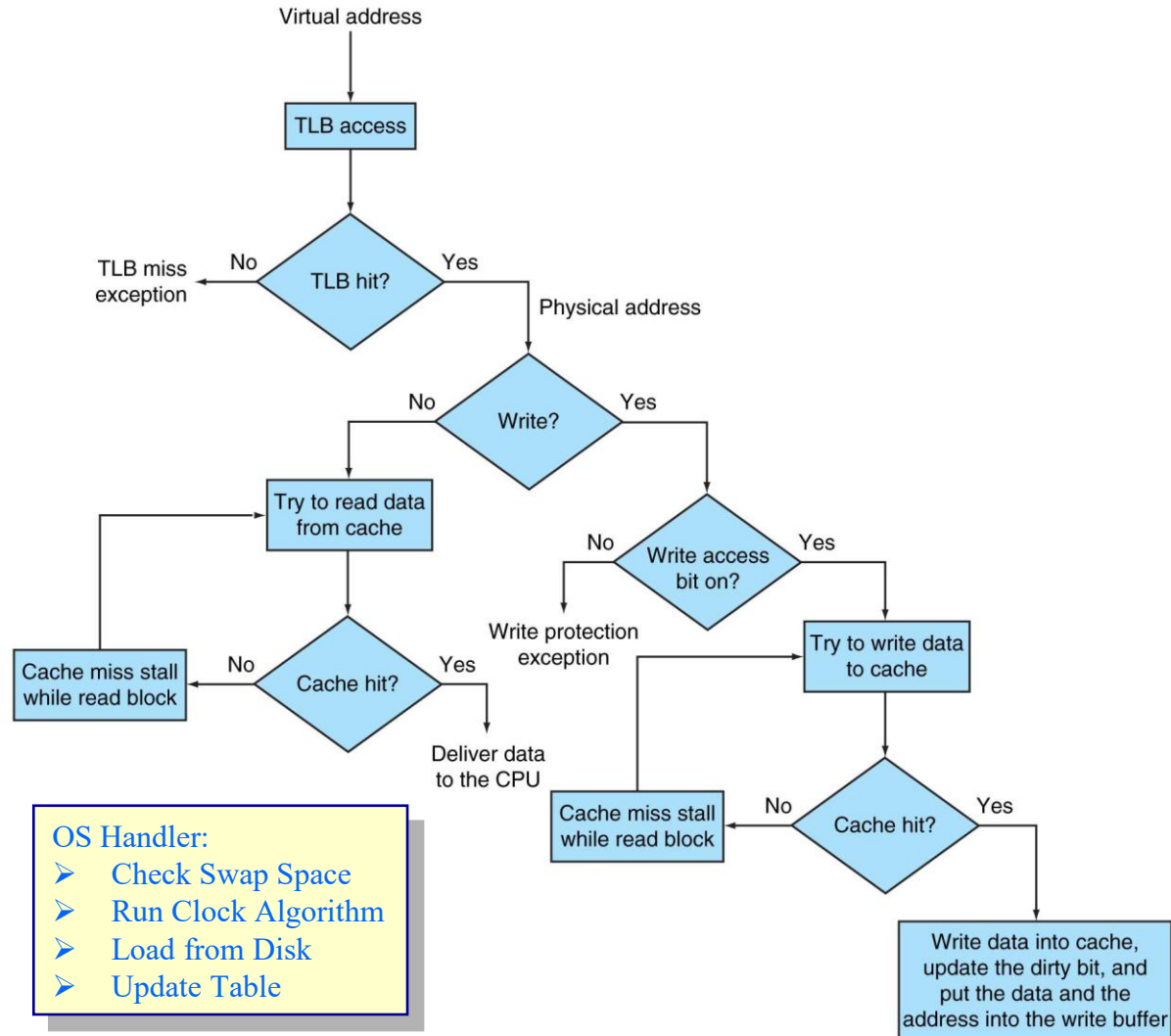
1	<i>e</i>
1	<i>b</i>
1	<i>a</i>
1	<i>c</i>

1	<i>d</i>
0	<i>b</i>
0	<i>a</i>
0	<i>c</i>

Disk Operations

- ❑ Disk writes take millions of cycles
 - Write through is impractical
 - Use **write-back**
 - **Dirty bit** in the page table entry set when page is written
 - CPU switches to other processes while disk writes to improve utilization
- ❑ Excessive faults → **thrashing**
 - OS constantly servicing page faults, little actual program progress
 - CPU utilization drops to near 0%, disk light is solid on
 - Cause: The active “working set” of current programs > physical RAM.
 - **Solution:** Kill a process or buy more RAM.

Summary: The Complete Flow for Memory Access



CONCLUSION

Key takeaways

- > Virtual Memory provides
 - Illusion of large memory & process isolation
- > Key mechanisms
 - Pages/Frames: Fixed-size memory blocks
 - Page Tables: Virtual-to-physical mapping
 - TLB: Fast translation cache
 - Demand Paging: Managing limited physical memory
 - HW+ OS: Handling page faults (page replacement)

NEXT LECTURE

[Flipped class] Interfacing with Peripherals

- > Pre-class
 - Homework
 - Self study
- > In class
 - Reinforcement/enrichment discussion
 - Quizzes
- > Post class
 - Homework
 - Consultation (if needed)